

> LAB 1

🔍 6 cells hidden

✓ LAB 2

Perceptron learning Algorithm for Implementing AND Logic Gate

```
import numpy as np
# for AND gate (Input/Output)
x_list = np.array([[0,0],[0,1],[1,0],[1,1]])
y_list = np.array([0,0,0,1])
epochs = 4
w,b = np.zeros(2),0

for _ in range(epochs):
    for x,y in zip(x_list,y_list):
        z = np.dot(x,w) + b
        ### condition area for y_pred
        y_pred = 1 if z >= 0 else 0
        E = y - y_pred
        print("error : ",E,"<- at epoch",_)
        w += E*x
        b += E
    print(w,b)
```

```
error : -1 <- at epoch 0
[0. 0.] -1
error : 0 <- at epoch 0
[0. 0.] -1
error : 0 <- at epoch 0
[0. 0.] -1
error : 1 <- at epoch 0
[1. 1.] 0
error : -1 <- at epoch 1
[1. 1.] -1
error : -1 <- at epoch 1
[1. 0.] -2
error : 0 <- at epoch 1
[1. 0.] -2
error : 1 <- at epoch 1
[2. 1.] -1
error : 0 <- at epoch 2
[2. 1.] -1
error : -1 <- at epoch 2
[2. 0.] -2
error : -1 <- at epoch 2
[1. 0.] -3
error : 1 <- at epoch 2
[2. 1.] -2
error : 0 <- at epoch 3
[2. 1.] -2
error : 0 <- at epoch 3
[2. 1.] -2
error : -1 <- at epoch 3
[1. 1.] -3
error : 1 <- at epoch 3
[2. 2.] -2
```

✓ Tasks to be performed

1. Change the epochs and observe the output
2. Change the random weights and bias and observe the convergence of the network.
3. Add logic to stop the training when the error is 'zero' for all the inputs.
4. Repeat the same process for OR gate
5. Try the process for XOR gate and show that the network will not stabilize (Converge).

task in order:

```
import numpy as np
# for AND gate (Input/Output)
x_list = np.array([[0,0],[0,1],[1,0],[1,1]])
y_list = np.array([0,0,0,1])
epochs = 10
w,b = np.zeros(2),0

for _ in range(epochs):
    for x,y in zip(x_list,y_list):
        z = np.dot(x,w) + b
        ### condition area for y_pred
        y_pred = 1 if z >= 0 else 0
        E = y - y_pred
        print("error : ",E,"<- at epoch",_)
        w += E*x
        b += E
    print(w,b)
    print("Azeez zaidi 418")
```

```
error : -1 <- at epoch 0
[0. 0.] -1
error : 0 <- at epoch 0
[0. 0.] -1
error : 0 <- at epoch 0
[0. 0.] -1
error : 1 <- at epoch 0
[1. 1.] 0
error : -1 <- at epoch 1
[1. 1.] -1
error : -1 <- at epoch 1
[1. 0.] -2
error : 0 <- at epoch 1
[1. 0.] -2
error : 1 <- at epoch 1
[2. 1.] -1
error : 0 <- at epoch 2
[2. 1.] -1
error : -1 <- at epoch 2
[2. 0.] -2
error : -1 <- at epoch 2
[1. 0.] -3
error : 1 <- at epoch 2
[2. 1.] -2
error : 0 <- at epoch 3
[2. 1.] -2
error : 0 <- at epoch 3
[2. 1.] -2
error : -1 <- at epoch 3
[1. 1.] -3
error : 1 <- at epoch 3
[2. 2.] -2
error : 0 <- at epoch 4
[2. 2.] -2
error : -1 <- at epoch 4
[2. 1.] -3
error : 0 <- at epoch 4
[2. 1.] -3
error : 0 <- at epoch 4
[2. 1.] -3
error : 0 <- at epoch 5
[2. 1.] -3
error : 0 <- at epoch 5
[2. 1.] -3
error : 0 <- at epoch 5
[2. 1.] -3
error : 0 <- at epoch 5
[2. 1.] -3
error : 0 <- at epoch 6
[2. 1.] -3
error : 0 <- at epoch 6
[2. 1.] -3
error : 0 <- at epoch 6
[2. 1.] -3
error : 0 <- at epoch 7
[2. 1.] -3
```

```

import numpy as np
# for AND gate (Input/Output)
x_list = np.array([[0,0],[0,1],[1,0],[1,1]])
y_list = np.array([0,0,0,1])
epochs = 10
w,b = np.array([2,5]),0
for _ in range(epochs):
    for x,y in zip(x_list,y_list):
        z = np.dot(x,w) + b
        ### condition area for y_pred
        y_pred = 1 if z >= 0 else 0
        E = y - y_pred
        print("error : ",E,"<- at epoch",_)
        w += E*x
        b += E
    print(w,b)
    print("Azeez 418")
    print("w=2,b=5")

```

```

Azeez 418
w=2,b=5
error :  0 <- at epoch 6
[1 3] -4
Azeez 418
w=2,b=5
error :  0 <- at epoch 6
[1 3] -4
Azeez 418
w=2,b=5
error :  0 <- at epoch 7
[1 3] -4
Azeez 418
w=2,b=5
error :  0 <- at epoch 7
[1 3] -4
Azeez 418
w=2,b=5
error :  0 <- at epoch 7
[1 3] -4
Azeez 418
w=2,b=5
error :  0 <- at epoch 7
[1 3] -4
Azeez 418
w=2,b=5
error :  0 <- at epoch 7
[1 3] -4
Azeez 418
w=2,b=5
error :  0 <- at epoch 8
[1 3] -4
Azeez 418
w=2,b=5
error :  0 <- at epoch 8
[1 3] -4
Azeez 418
w=2,b=5
error :  0 <- at epoch 8
[1 3] -4
Azeez 418
w=2,b=5
error :  0 <- at epoch 8
[1 3] -4
Azeez 418
w=2,b=5
error :  0 <- at epoch 9
[1 3] -4
Azeez 418
w=2,b=5
error :  0 <- at epoch 9
[1 3] -4
Azeez 418
w=2,b=5
error :  0 <- at epoch 9
[1 3] -4
Azeez 418
w=2,b=5
error :  0 <- at epoch 9
[1 3] -4
Azeez 418
w=2,b=5

```

```

import numpy as np
# for AND gate (Input/Output)
x_list = np.array([[0,0],[0,1],[1,0],[1,1]])

```

```

y_list = np.array([0,0,0,1])
epochs = 10 # Increased epochs to ensure convergence without fixed iterations
w,b = np.zeros(2),0

for _ in range(epochs):
    errors_in_epoch = False # Flag to track errors in the current epoch
    for x,y in zip(x_list,y_list):
        z = np.dot(x,w) + b
        y_pred = 1 if z >= 0 else 0
        E = y - y_pred
        print("error : ",E,"is at epoch",_)
        if E != 0:
            w += E*x
            b += E
            errors_in_epoch = True
    print(w,b)
if not errors_in_epoch:
    print(f"Training stopped at epoch {_} as error has become zero for all the following inputs.")
    break

```

```

error : -1 is at epoch 0
[0. 0.] -1
error : 0 is at epoch 0
[0. 0.] -1
error : 0 is at epoch 0
[0. 0.] -1
error : 1 is at epoch 0
[1. 1.] 0
error : -1 is at epoch 1
[1. 1.] -1
error : -1 is at epoch 1
[1. 0.] -2
error : 0 is at epoch 1
[1. 0.] -2
error : 1 is at epoch 1
[2. 1.] -1
error : 0 is at epoch 2
[2. 1.] -1
error : -1 is at epoch 2
[2. 0.] -2
error : -1 is at epoch 2
[1. 0.] -3
error : 1 is at epoch 2
[2. 1.] -2
error : 0 is at epoch 3
[2. 1.] -2
error : 0 is at epoch 3
[2. 1.] -2
error : -1 is at epoch 3
[1. 1.] -3
error : 1 is at epoch 3
[2. 2.] -2
error : 0 is at epoch 4
[2. 2.] -2
error : -1 is at epoch 4
[2. 1.] -3
error : 0 is at epoch 4
[2. 1.] -3
error : 0 is at epoch 4
[2. 1.] -3
error : 0 is at epoch 5
[2. 1.] -3
error : 0 is at epoch 5
[2. 1.] -3
error : 0 is at epoch 5
[2. 1.] -3
error : 0 is at epoch 5
[2. 1.] -3
Training stopped at epoch 5 as error has become zero for all the following inputs.

```

```

import numpy as np
# for OR gate (Input/Output)
x_list = np.array([[0,0],[0,1],[1,0],[1,1]])
y_list = np.array([0,1,1,1])
epochs = 10
w,b = np.zeros(2),0

for _ in range(epochs):
    for x,y in zip(x_list,y_list):
        z = np.dot(x,w) + b
        ### condition area for y_pred

```

```

y_pred = 1 if z >= 0 else 0
E = y - y_pred
print("error : ",E,"<- at epoch",_)
w += E*x
b += E
print(w,b)
print("Azeez zaidi 418")

```

```

error : -1 <- at epoch 0
[0. 0.] -1
Azeez zaidi 418
error : 1 <- at epoch 0
[0. 1.] 0
Azeez zaidi 418
error : 0 <- at epoch 0
[0. 1.] 0
Azeez zaidi 418
error : 0 <- at epoch 0
[0. 1.] 0
Azeez zaidi 418
error : -1 <- at epoch 1
[0. 1.] -1
Azeez zaidi 418
error : 0 <- at epoch 1
[0. 1.] -1
Azeez zaidi 418
error : 1 <- at epoch 1
[1. 1.] 0
Azeez zaidi 418
error : 0 <- at epoch 1
[1. 1.] 0
Azeez zaidi 418
error : -1 <- at epoch 2
[1. 1.] -1
Azeez zaidi 418
error : 0 <- at epoch 2
[1. 1.] -1
Azeez zaidi 418
error : 0 <- at epoch 2
[1. 1.] -1
Azeez zaidi 418
error : 0 <- at epoch 2
[1. 1.] -1
Azeez zaidi 418
error : 0 <- at epoch 3
[1. 1.] -1
Azeez zaidi 418
error : 0 <- at epoch 3
[1. 1.] -1
Azeez zaidi 418
error : 0 <- at epoch 3
[1. 1.] -1
Azeez zaidi 418
error : 0 <- at epoch 3
[1. 1.] -1
Azeez zaidi 418
error : 0 <- at epoch 4
[1. 1.] -1
Azeez zaidi 418
error : 0 <- at epoch 4
[1. 1.] -1
Azeez zaidi 418
error : 0 <- at epoch 4
[1. 1.] -1
Azeez zaidi 418
error : 0 <- at epoch 4

```

```

import numpy as np
# for OR gate (Input/Output)
x_list = np.array([[0,0],[0,1],[1,0],[1,1]])
y_list = np.array([0,1,1,1])
epochs = 10
w,b = np.array([2,5]),0
for _ in range(epochs):
    for x,y in zip(x_list,y_list):
        z = np.dot(x,w) + b
        ### condition area for y_pred
        y_pred = 1 if z >= 0 else 0
        E = y - y_pred
        print("error : ",E,"<- at epoch",_)
        w += E*x
        b += E

```

```

print(w,b)
print("Azeez 418")

error : -1 <- at epoch 0
[2 5] -1
Azeez 418
error : 0 <- at epoch 0
[2 5] -1
Azeez 418
error : 0 <- at epoch 0
[2 5] -1
Azeez 418
error : 0 <- at epoch 0
[2 5] -1
Azeez 418
error : 0 <- at epoch 1
[2 5] -1
Azeez 418
error : 0 <- at epoch 1
[2 5] -1
Azeez 418
error : 0 <- at epoch 1
[2 5] -1
Azeez 418
error : 0 <- at epoch 1
[2 5] -1
Azeez 418
error : 0 <- at epoch 2
[2 5] -1
Azeez 418
error : 0 <- at epoch 2
[2 5] -1
Azeez 418
error : 0 <- at epoch 2
[2 5] -1
Azeez 418
error : 0 <- at epoch 2
[2 5] -1
Azeez 418
error : 0 <- at epoch 3
[2 5] -1
Azeez 418
error : 0 <- at epoch 3
[2 5] -1
Azeez 418
error : 0 <- at epoch 3
[2 5] -1
Azeez 418
error : 0 <- at epoch 3
[2 5] -1
Azeez 418
error : 0 <- at epoch 4
[2 5] -1
Azeez 418
error : 0 <- at epoch 4
[2 5] -1
Azeez 418
error : 0 <- at epoch 4

```

```

import numpy as np
# for OR gate (Input/Output)
x_list = np.array([[0,0],[0,1],[1,0],[1,1]])
y_list = np.array([0,1,1,1])
epochs = 10 # Increased epochs to ensure convergence without fixed iterations
w,b = np.zeros(2),0

for _ in range(epochs):
    errors_in_epoch = False # Flag to track errors in the current epoch
    for x,y in zip(x_list,y_list):
        z = np.dot(x,w) + b
        y_pred = 1 if z >= 0 else 0
        E = y - y_pred
        print("error : ",E,"is at epoch",_)
        if E != 0:
            w += E*x
            b += E
            errors_in_epoch = True
        print(w,b)
    if not errors_in_epoch:

```

```

print(f"Training stopped at epoch {_} as error has become zero for all the following inputs.")
break

error : -1 is at epoch 0
[0. 0.] -1
error : 1 is at epoch 0
[0. 1.] 0
error : 0 is at epoch 0
[0. 1.] 0
error : 0 is at epoch 0
[0. 1.] 0
error : -1 is at epoch 1
[0. 1.] -1
error : 0 is at epoch 1
[0. 1.] -1
error : 1 is at epoch 1
[1. 1.] 0
error : 0 is at epoch 1
[1. 1.] 0
error : -1 is at epoch 2
[1. 1.] -1
error : 0 is at epoch 2
[1. 1.] -1
error : 0 is at epoch 2
[1. 1.] -1
error : 0 is at epoch 2
[1. 1.] -1
error : 0 is at epoch 3
[1. 1.] -1
error : 0 is at epoch 3
[1. 1.] -1
error : 0 is at epoch 3
[1. 1.] -1
error : 0 is at epoch 3
[1. 1.] -1
Training stopped at epoch 3 as error has become zero for all the following inputs.

```

```

import numpy as np
# for OR gate (Input/Output)
x_list = np.array([[0,0],[0,1],[1,0],[1,1]])
y_list = np.array([0,1,1,0])
epochs = 10 # Increased epochs to ensure convergence without fixed iterations
w,b = np.zeros(2),0

for _ in range(epochs):
    errors_in_epoch = False # Flag to track errors in the current epoch
    for x,y in zip(x_list,y_list):
        z = np.dot(x,w) + b
        y_pred = 1 if z >= 0 else 0
        E = y - y_pred
        print("error : ",E,"is at epoch",_)
        if E != 0:
            w += E*x
            b += E
            errors_in_epoch = True
    print(w,b)
if not errors_in_epoch:
    print(f"Training stopped at epoch {_} as error has become zero for all the following inputs.")
    break

```

```

error : -1 is at epoch 0
[0. 0.] -1
error : 1 is at epoch 0
[0. 1.] 0
error : 0 is at epoch 0
[0. 1.] 0
error : -1 is at epoch 0
[-1.  0.] -1
error : 0 is at epoch 1
[-1.  0.] -1
error : 1 is at epoch 1
[-1.  1.] 0
error : 1 is at epoch 1
[0.  1.] 1
error : -1 is at epoch 1
[-1.  0.] 0
error : -1 is at epoch 2
[-1.  0.] -1
error : 1 is at epoch 2
[-1.  1.] 0
error : 1 is at epoch 2

```

```
[0. 1.] 1
error : -1 is at epoch 2
[-1. 0.] 0
error : -1 is at epoch 3
[-1. 0.] -1
error : 1 is at epoch 3
[-1. 1.] 0
error : 1 is at epoch 3
[0. 1.] 1
error : -1 is at epoch 3
[-1. 0.] 0
error : -1 is at epoch 4
[-1. 0.] -1
error : 1 is at epoch 4
[-1. 1.] 0
error : 1 is at epoch 4
[0. 1.] 1
error : -1 is at epoch 4
[-1. 0.] 0
error : -1 is at epoch 5
[-1. 0.] -1
error : 1 is at epoch 5
[-1. 1.] 0
error : 1 is at epoch 5
[0. 1.] 1
error : -1 is at epoch 5
[-1. 0.] 0
error : -1 is at epoch 6
[-1. 0.] -1
error : 1 is at epoch 6
[-1. 1.] 0
error : 1 is at epoch 6
[0. 1.] 1
error : -1 is at epoch 6
[-1. 0.] 0
error : -1 is at epoch 7
```